

WEBSITE SCRAPING AND ANALYSIS OF

INFORMATION SECURITY



REMOTE|OK

PRESENTED BY:

ALAA AZOUZI

SARRA ALIOUA

NOUHA BOUKHRIS

MARIEM CHAMMEM

Supervised by:

Dr. Manel Abdelkader

Academic Year: 2025-2026

TUNIS BUSINESS SCHOOL

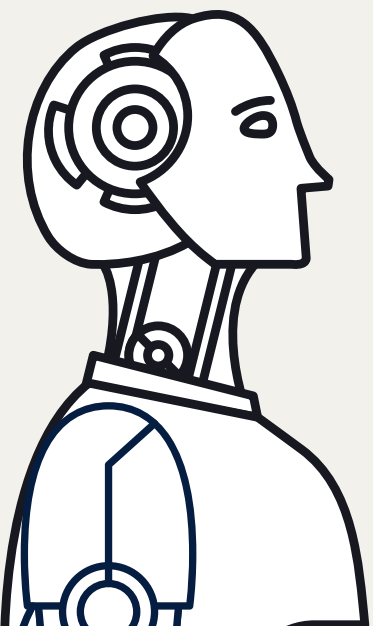


Table of Contents

01

Introduction

Company • Mission • Tools

03

Execution Process

Data Collection • Cleaning • Visualization

02

System Design

Overview • Components • DFDs • Architecture

04

Conclusion

Key Outcomes & Future Extensions

01

Introduction

1.1 Company Introduction

1.2 Context

1.3 Project Mission

1.4 Tools & Software Used

1.1 Company Introduction



RemoteOK Platform

RemoteOK is a global remote job platform that aggregates job opportunities for developers, designers, data professionals, and digital workers.

It is widely used by startups and tech companies to publish remote-first positions.

🔑 Key Differentiator :

Unlike traditional job boards, RemoteOK provides a public API that exposes structured job data — making it ideal for data analysis, automation, and job market research.

Global

Remote Platform

Public

API Access

Structured

JSON Data

1.2 Context

Ethical Scraping

Web scraping can violate terms of service, overload servers, or expose private data. This project uses RemoteOK's official public API, ensuring legal and ethical data access.

Data Integrity

Ensuring the pipeline produces trustworthy, unaltered data is a core principle of information assurance. Our cleaning process enforces validation, deduplication, and standardization to guarantee data integrity.

Privacy by Design

The dataset contains no personal user data only publicly listed job postings. No credentials, tokens, or authentication are required or stored.

 **Key takeaway:** This project demonstrates responsible data collection aligned with information security principles: confidentiality, integrity, and availability (CIA triad).

1.3 Project Mission

Design and implement a job data extraction pipeline that delivers actionable market intelligence.

Collect

Remote job listings from the RemoteOK structured API

Filter

Jobs by relevant criteria: API roles, tech stack, keywords

Clean

Organize raw data into a clean, structured dataset

Export

Results in CSV format for further analysis

1.4 Tools & Software Used



python™

Core scripting & data processing



Version control & collaboration

BeautifulSoup

HTML parsing library



draw.io

System diagrams & architecture



Power BI

Data visualization & dashboards



pandas

Data manipulation & analysis

2. System Design

02

System Design

2.1 System Overview

2.2 Component Breakdown

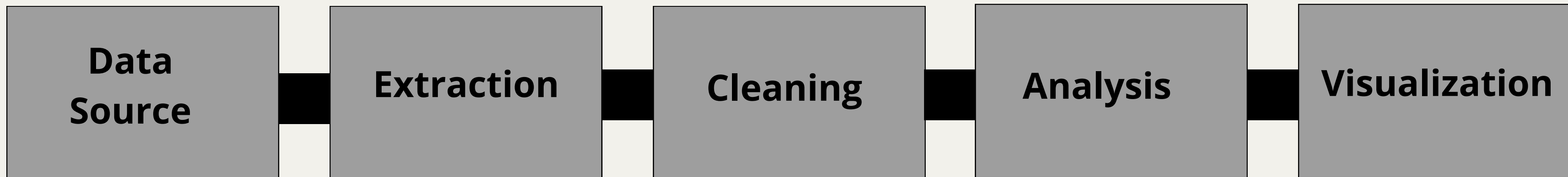
2.3 Data Flow Diagrams

2.4 Exchange Messages & Data

2.5 System Architecture

2.1 System Overview

A job market data pipeline that automatically collects, processes, and visualizes remote job listings from an external API.



Structured Data Engineering Workflow

Design Principles

API-based extraction: Uses RemoteOK's public JSON API instead of fragile HTML scraping

Modular architecture: Separate scripts for scraping (`remoteok_scraper.py`) and cleaning (`remoteok_cleaner.py`)

Reproducible pipeline: Each phase outputs a CSV file that feeds the next stage

2.2 Component Breakdown

API Client

remoteok_scraper.py

HTTP GET to remoteok.com/api with Mozilla/5.0 User-Agent. Validates status 200.

JSON Parser

remoteok_scraper.py

Iterates job objects (skipping metadata). Extracts: url, title, company, location, salary, date, skills.

DataFrame Builder

remoteok_scraper.py

Converts dict list to pandas DataFrame. Previews data, saves raw output as jobs_data.csv.

2.2 Component Breakdown

Data Cleaning Module

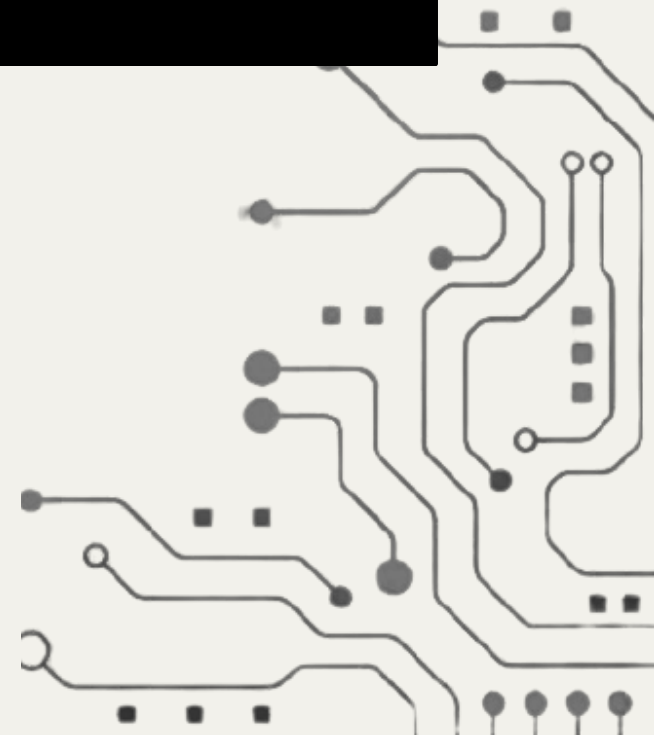
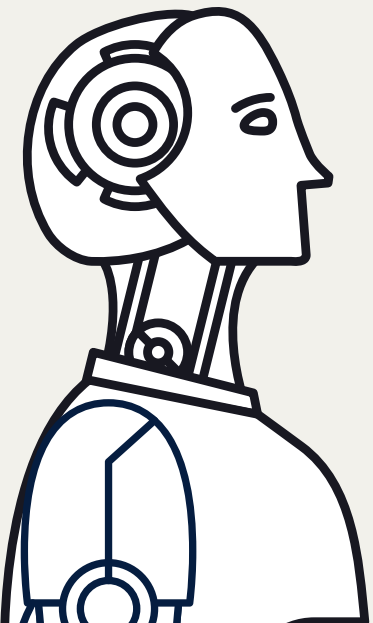
remoteok_scraper.py

Drops nulls, removes duplicates, standardises text, parses salary midpoint, converts dates.

Visualisation Layer

Power BI / Excel

Loads jobs_final_cleaned.csv. Produces dashboards: top companies, skills, salary, posting trends.



Import Libraries

requests · pandas · os



HTTP GET Request

remoteok.com/api + User-Agent: Mozilla/5.0



Validate Response

Status 200 → continue · else raise_for_status()



data[1:]



Structure Each Job Entry

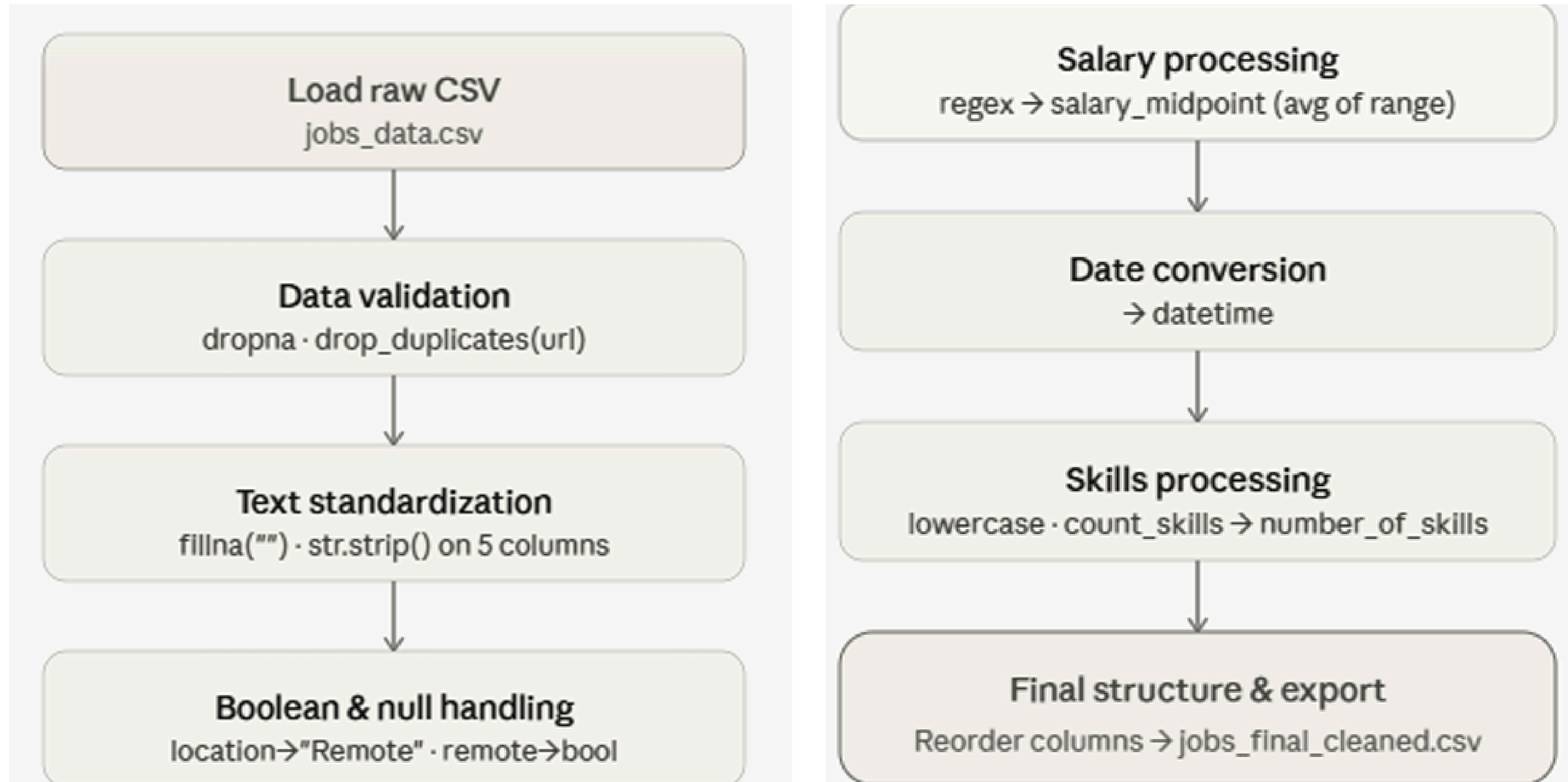
url · title · company · location · salary · date · skills



Build DataFrame & Export

pd.DataFrame(jobs) → jobs_data.csv

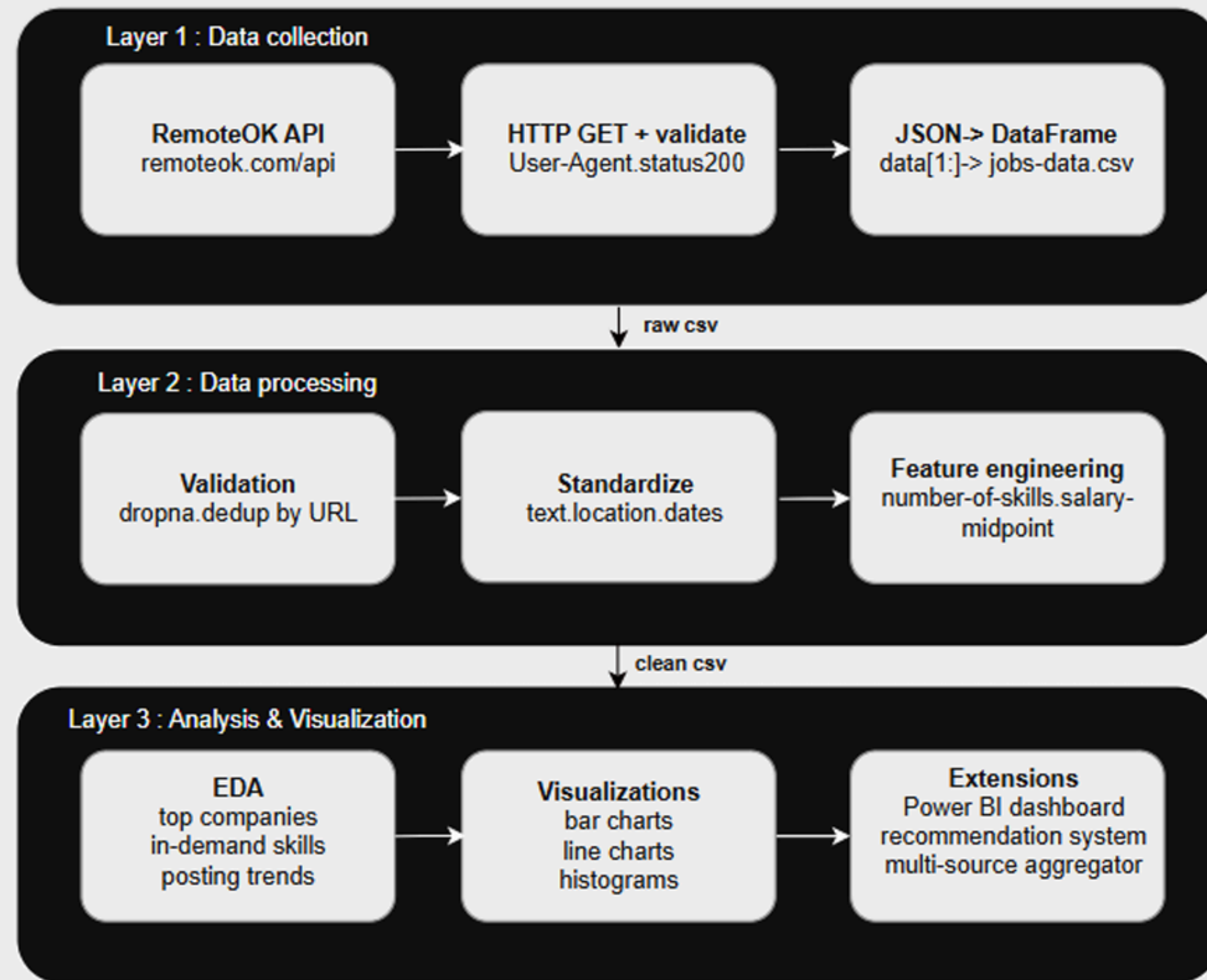
2.3 Data Flow — Cleaning Phase



2.4 Exchange Messages & Data Fields

Field Name	Source	Type	Description
url	remoteok.com/api	string	Direct link to the job posting page
title	remoteok.com/api	string	Job position title (e.g. Data Analyst)
company	remoteok.com/api	string	Name of the hiring company
location	remoteok.com/api	string	Location — defaults to 'Remote' if absent
salary	remoteok.com/api	string	Raw salary string as listed on the platform
date_posted	remoteok.com/api	string	Date the job was published (ISO format)
skills (tags)	remoteok.com/api	string	List of required skills/technologies
remote	Derived	boolean	True if position is fully remote
number_of_skills	Derived	integer	Count of skills listed per job posting

2.5 System Architecture



3. Execution Process

03

Execution Process

3.1 Data Collection & Extraction

3.2 Data Cleaning & Transformation

3.3 Data Visualization & Insights

3.4 Dashboard Charts & Insights

3.5 Limitations

3.1 Data Collection & Extraction (Steps 1-3)

1 Import Libraries & Set API Request

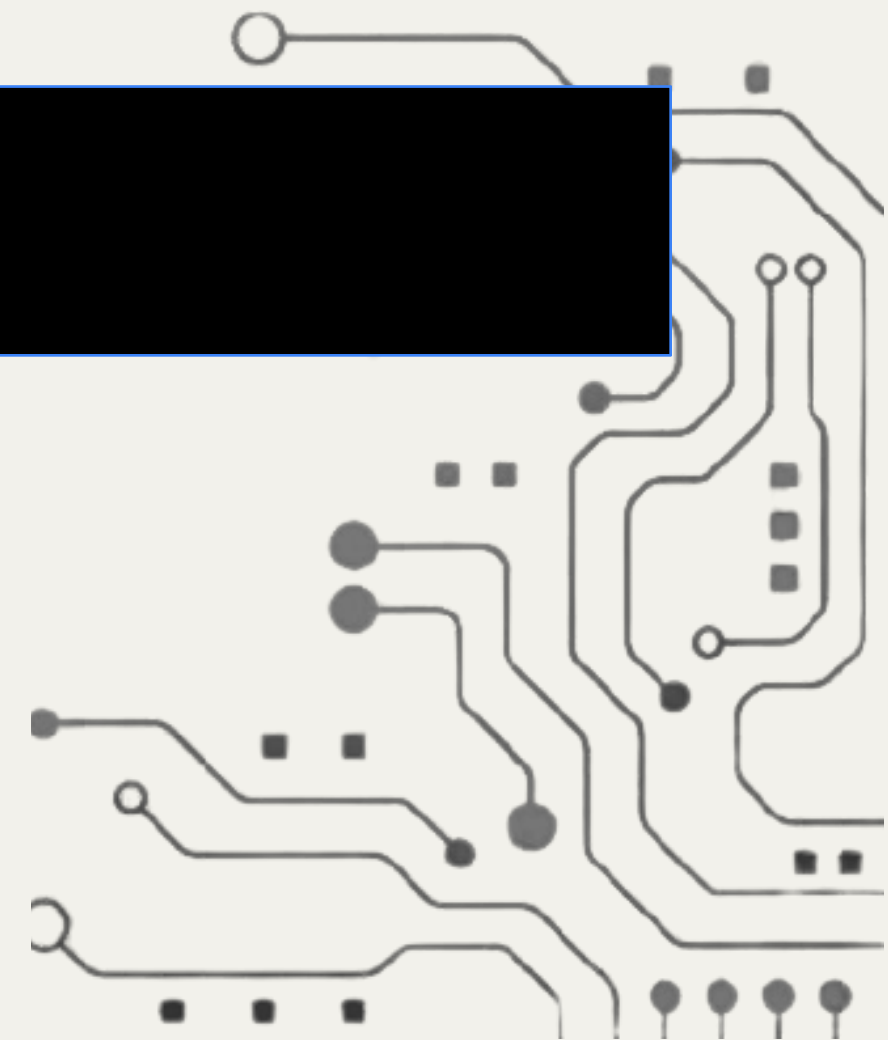
```
import requests
import pandas as pd
url = "https://remoteok.com/api"
headers = {"User-Agent": "Mozilla/5.0"}
```

2 Response Validation

```
response = requests.get(url, headers=headers)
print("Status code:", response.status_code)
response.raise_for_status()
```

3 JSON Parsing

- Response converted to Python object using `response.json()`
- First element is metadata — `skipped (data[1:])`



3.1 Data Collection & Extraction (Steps 4-6)

4 Data Structuring — Extracted Fields

url

title

company

location (→Remote)

salary

date_posted

skills (CSV string)

```
jobs = []
for job in data[1:]:
    jobs.append({
        "url": job.get("url"), "title": job.get("position"),
        "location": job.get("location") or "Remote",
        "skills": ", ".join(job.get("tags", []))
    })
```

5 DataFrame Creation

Convert dict list → pandas DataFrame · Print df.head()

```
df = pd.DataFrame(jobs) · df.to_csv(output_path, index=False, encoding="utf-8-sig")
```

6 Data Export

Save as jobs_data.csv · Encoding: UTF-8-sig

3.2 Data Cleaning & Transformation (Steps 1-4)

1 Data Loading

```
df = pd.read_csv("jobs_data.csv", encoding="utf-8-sig")
```

2 Data Validation — Drop nulls & duplicates

```
df.dropna(subset=["title","company"], how="all", inplace=True)  
df.drop_duplicates(subset=["url"], keep="first", inplace=True)
```

3 Text Standardization

```
text_columns = ["title","company","location","job_type","skills"]  
for col in text_columns:  
    df[col] = df[col].fillna("").astype(str).str.strip()
```

4 Boolean & Null Handling

```
df["location"] = df["location"].replace("", "Remote")  
df["remote"] = df["remote"].fillna(True).astype(bool)
```

3.2 Data Cleaning & Transformation (Steps 5-8)

5 Salary Processing — Extract Numeric Midpoint

```
def parse_salary_midpoint(salary):
    numbers = re.findall(
        r"\d+", salary.replace(",", ""))
    if len(numbers) >= 2: return (float(numbers[0]) + float(numbers[1])) / 2
    elif len(numbers) == 1: return float(numbers[0]) # single value
    else: return None
```

6 Date Conversion

`pd.to_datetime()` applied to `date_posted` for time-based analysis

7 Skills Processing

Lowercase all skills • `count_skills()` → `number_of_skills` column

8 Final Dataset Structuring & Export

Reorder 9 columns → `jobs_final_cleaned.csv` • `encoding="utf-8-sig"`

3.3 Data Visualization & Insights

99

Total available jobs

95

Total companies

30

Distinct Locations



Filters Panel

Slice by Company, Location, and Date Posted for targeted analysis



Jobs Posted Over Time

Line chart: April 18–25. Peak ~20 postings on April 22, suggesting mid-period hiring burst



Skills Distribution

Histogram: Most jobs require 5–15 skills; peak at 10–12, showing moderate-to-high skill demand



Top 6 Job Locations

Pie chart: Remote = 86.67%. Canada, Los Angeles, US each at 2.67%



Job Type Distribution

Donut chart: 100% of 99 jobs are remote — confirming the remote-first market

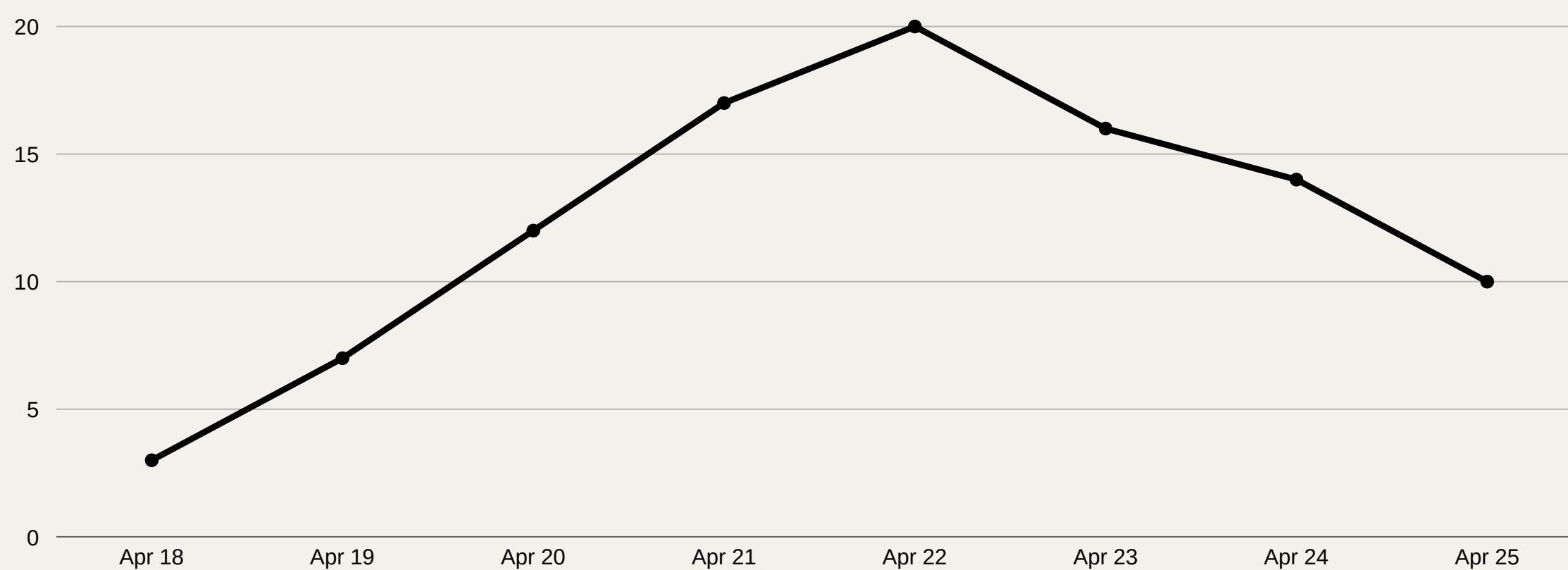


Companies by Remote Jobs

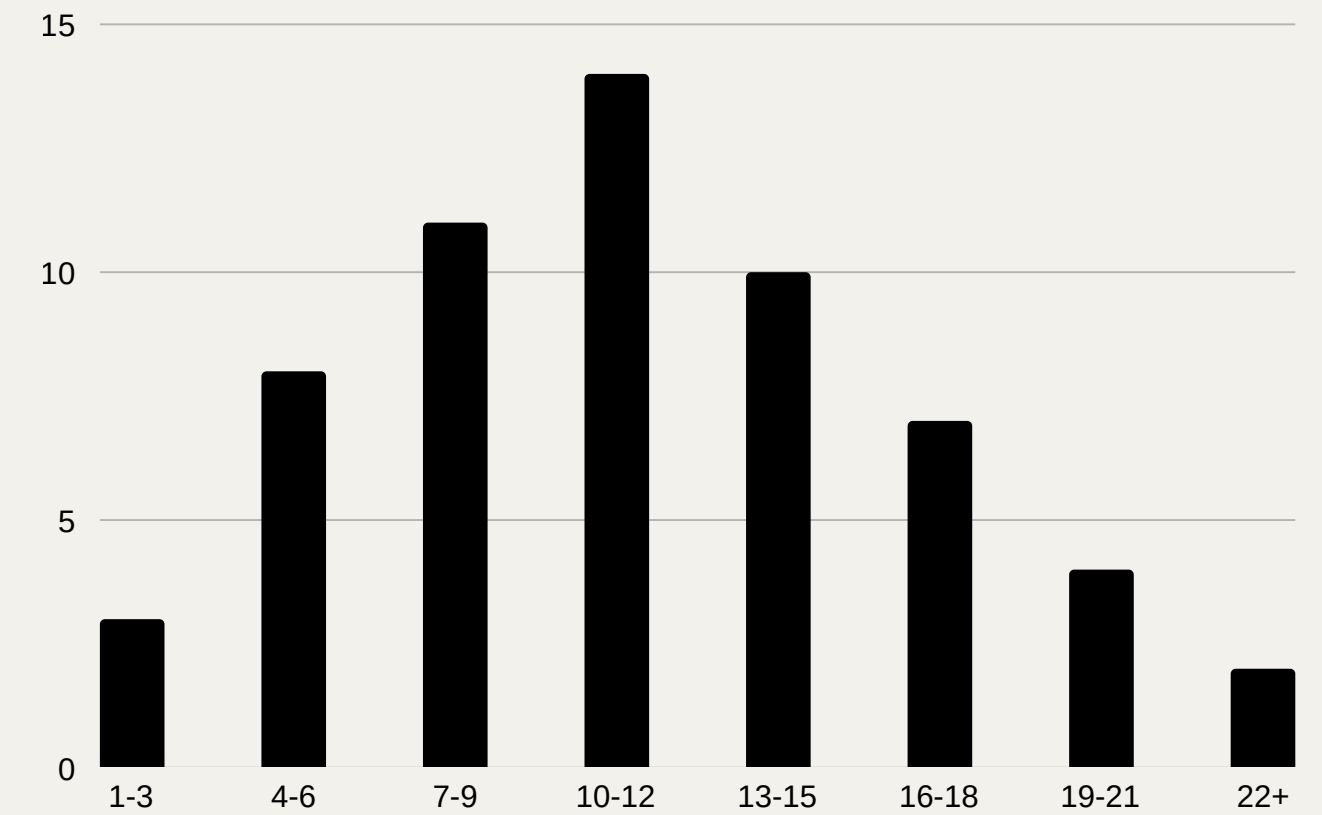
Bar chart: Pivotal Health, Adsweve Inc., Maneuver Marketing lead with ~3 postings each

3.4 Dashboard Charts & Insights

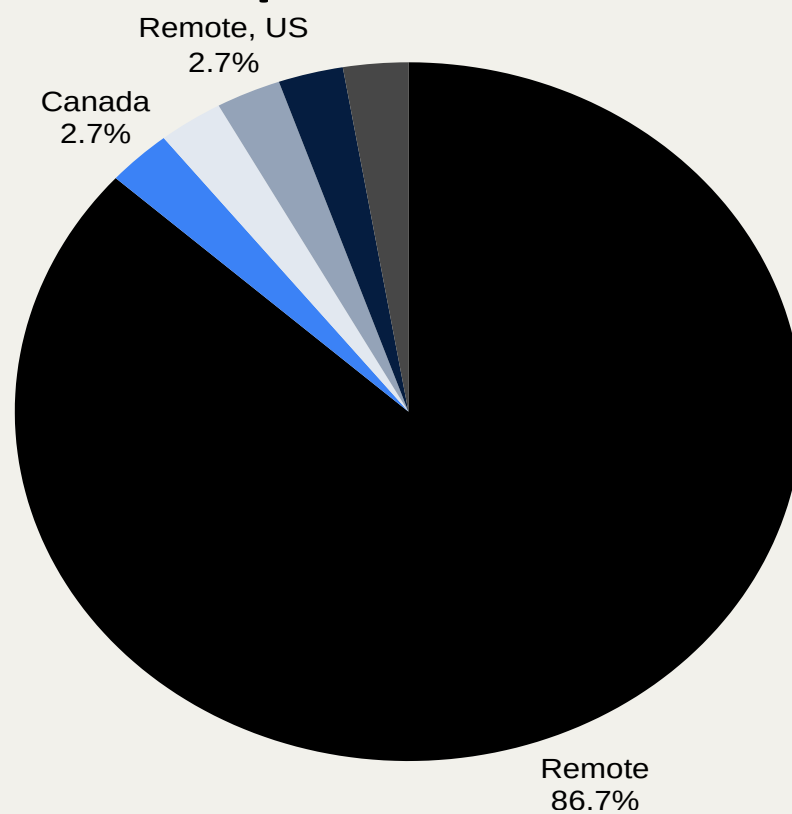
Number of Jobs Posted Over Time



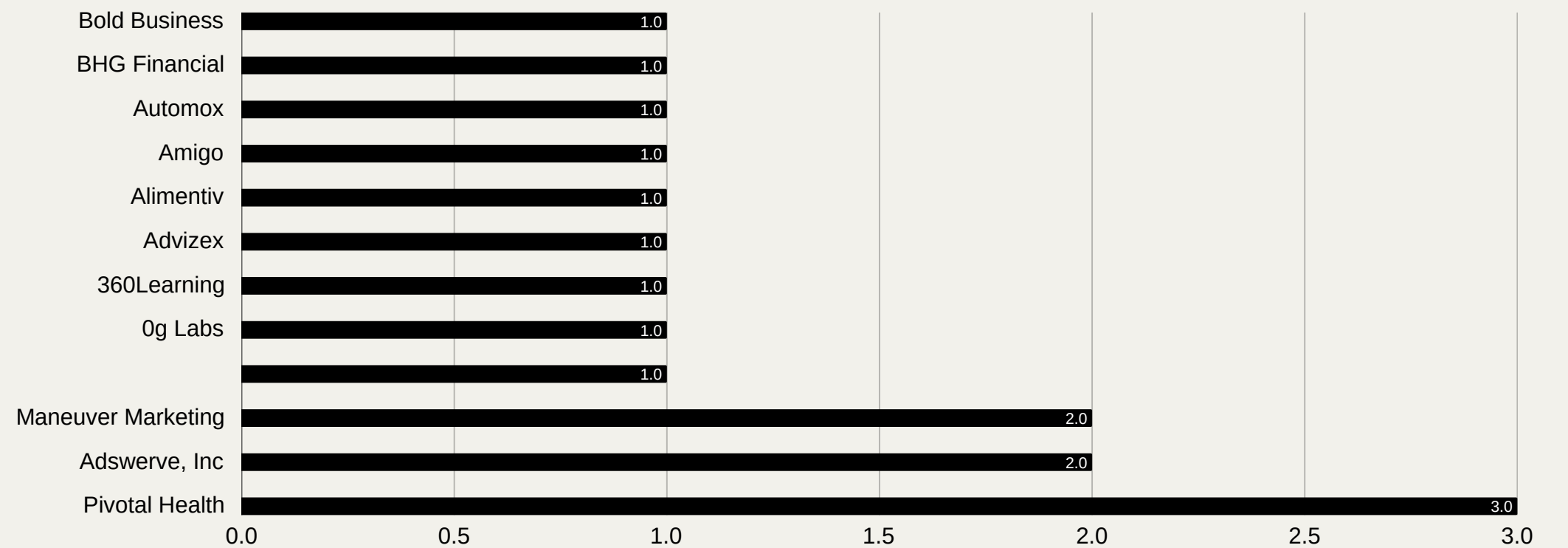
Skills Required per Job



Top 6 Job Locations



Companies by Number of Remote Jobs



3.5 Limitation & Observations

Dataset Size

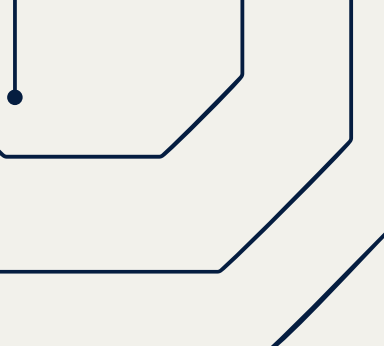
- The dataset contains only 99 job postings collected over a narrow window (April 18–25, 2026).
- Results may not represent long-term or seasonal hiring trends on the platform.

Salary Coverage

- The majority of job postings on RemoteOK do not include salary information. The salary_midpoint field is therefore null for most records, limiting salary-based analysis.

Single Source Limitation

- Data comes exclusively from RemoteOK. Findings reflect this platform's specific audience (startups, tech-first companies) and should not be generalized to the broader job market.



CONCLUSION

This project demonstrates a complete data extraction and transformation pipeline using real-world job market data.

Key outcomes:

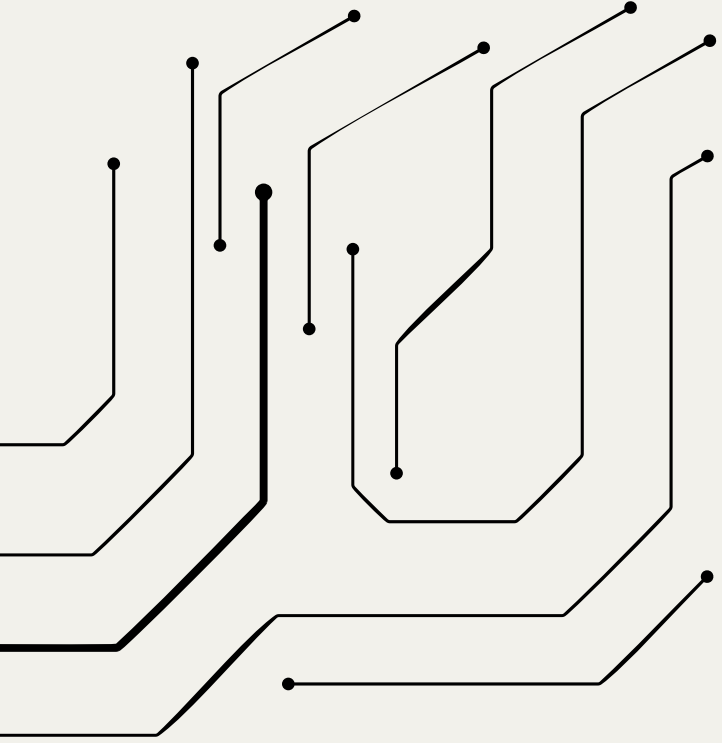
- Successfully connected to a live job API
- Extracted structured job data without web scraping instability
- Filtered and organized data into meaningful insights
- Built a reusable workflow for job market analysis

This approach highlights a key industry principle:
Modern data extraction is shifting from HTML scraping to API-based data engineering.

The project can be extended into:

- a job analytics dashboard
- a recommendation system
- or a multi-source job aggregator platform





THANK YOU !

Supervised by Dr. Manel Abdelkader · IT 360 – Tunis Business School · 2025-2026

r|OK

